

Module 2 – Hooks & Context

1. useState & useEffect

useState Hook

React Hook used to manage state in functional components.

Key Points :

- Stores dynamic data
- Returns state + updater function
- Triggers re-render on update

Example :

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

useEffect Hook

Used to handle side effects in components.

Examples of Side Effects :

- API calls
- DOM updates
- Timers

Example :

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Component Mounted");
}, []);
```

Key Points:

- Runs after render
- Dependency array controls execution

- Empty array = run once
-

2. Custom Hooks

What are Custom Hooks?

Reusable functions that contain logic using React hooks.

Why Use Them?

- Avoid code duplication
- Improve readability
- Reuse logic across components

Example

```
function useCounter() {  
  const [count, setCount] = useState(0);  
  
  const increment = () => setCount(count + 1);  
  
  return { count, increment };  
}
```

3. Context API Basics

What is Context API?

Used to share data globally without passing props manually.

Problem Solved :

- Avoids **prop drilling**
-

Creating Context

Example

```
import { createContext } from "react";  
  
const UserContext = createContext();
```

Using Context

Example :

```
<UserContext.Provider value="Ali">
  <Child />
</UserContext.Provider>
import { useContext } from "react";

const value = useContext(UserContext);
```

4. Global State

What is Global State?

Data shared across multiple components.

Examples

- User authentication
- Theme settings
- Language preferences

Key Points

- Managed using Context API or libraries
 - Avoid excessive prop passing
-

5. useReducer Hook

What is useReducer?

Used for managing complex state logic.

When to Use :

- Multiple state values
 - Complex updates
 - Large applications
-

Example:

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "increment":
```

```
    return { count: state.count + 1 };  
  default:  
    return state;  
}  
}  
  
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

6. Key Concepts to Remember

- Hooks simplify state and lifecycle management
 - useState handles simple state
 - useEffect handles side effects
 - Custom hooks reuse logic
 - Context API avoids prop drilling
 - useReducer manages complex state
-

7. Summary

Hooks and Context API are essential for modern React development. They allow cleaner, more scalable, and maintainable code by simplifying state management and enabling global data sharing across components.